Main.c

```c
#include <stdio.h>
#include <stdlib.h>
void merge(int arr[], int left, int mid, int right) {
 int n1 = mid - left + 1;
 int n2 = right - mid;
 int *L = (int *)malloc(n1 * sizeof(int));
 int *R = (int *)malloc(n2 * sizeof(int));
 for (int i = 0; i < n1; i++) L[i] = arr[left + i];
 for (int j = 0; j < n2; j++) R[j] = arr[mid + 1 + j];
 int i = 0, j = 0, k = left;
 while (i < n1 && j < n2) {
 if (L[i] <= R[j]) arr[k++] = L[i++];
 else arr[k++] = R[j++];
 }
 while (i < n1) arr[k++] = L[i++];
 while (j < n2) arr[k++] = R[j++];
 free(L);
 free(R);
}
void mergeSort(int arr[], int left, int right) {
 if (left < right) {
 int mid = left + (right - left) / 2;
 mergeSort(arr, left, mid);
 mergeSort(arr, mid + 1, right);
 merge(arr, left, mid, right);
 }
}
int main() {
 int n;
 scanf("%d", &n);
 int *marks = (int *)malloc(n * sizeof(int));
 for (int i = 0; i < n; i++) scanf("%d", &marks[i]);
 mergeSort(marks, 0, n - 1);
 for (int i = 0; i < n; i++) {
 printf("%d", marks[i]);
 if (i != n - 1) printf(" ");
 }
 printf("\n");
 free(marks);
 return 0;
}
```

Console I/O AI Agent 1 ms

STDIN

```
6
78 45 92 34 67 81
```

Output:

```
34 45 67 78 81 92
```

Main.c

```c
#include <stdio.h>
#include <stdlib.h>
void merge(int arr[], int left, int mid, int right) {
 int n1 = mid - left + 1;
 int n2 = right - mid;
 int *L = (int *)malloc(n1 * sizeof(int));
 int *R = (int *)malloc(n2 * sizeof(int));
 for (int i = 0; i < n1; i++) L[i] = arr[left + i];
 for (int j = 0; j < n2; j++) R[j] = arr[mid + 1 + j];
 int i = 0, j = 0, k = left;
 while (i < n1 && j < n2) {
 if (L[i] <= R[j]) arr[k++] = L[i++];
 else arr[k++] = R[j++];
 }
 while (i < n1) arr[k++] = L[i++];
 while (j < n2) arr[k++] = R[j++];
 free(L);
 free(R);
}
void mergeSort(int arr[], int left, int right) {
 if (left < right) {
 int mid = left + (right - left) / 2;
 mergeSort(arr, left, mid);
 mergeSort(arr, mid + 1, right);
 merge(arr, left, mid, right);
 }
}
int main() {
 int n;
 scanf("%d", &n);
 int *prices = (int *)malloc(n * sizeof(int));
 for (int i = 0; i < n; i++) scanf("%d", &prices[i]);
 mergeSort(prices, 0, n - 1);
 for (int i = 0; i < n; i++) {
 printf("%d", prices[i]);
 if (i != n - 1) printf(" ");
 }
 printf("\n");
 free(prices);
 return 0;
}
```

Console I/O AI Agent 2 ms

STDIN

```
5
2500 1500 4000 3000 1000
```

Output:

```
1000 1500 2500 3000 4000
```

Main.c

```c
#include <stdio.h>
#include <stdlib.h>
void merge(long long arr[], int left, int mid, int right) {
 int n1 = mid - left + 1;
 int n2 = right - mid;
 long long *L = (long long *)malloc(n1 * sizeof(long long));
 long long *R = (long long *)malloc(n2 * sizeof(long long));
 for (int i = 0; i < n1; i++) L[i] = arr[left + i];
 for (int j = 0; j < n2; j++) R[j] = arr[mid + 1 + j];
 int i = 0, j = 0, k = left;
 while (i < n1 && j < n2) {
 if (L[i] <= R[j]) arr[k++] = L[i++];
 else arr[k++] = R[j++];
 }
 while (i < n1) arr[k++] = L[i++];
 while (j < n2) arr[k++] = R[j++];
 free(L);
 free(R);
}
void mergeSort(long long arr[], int left, int right) {
 if (left < right) {
 int mid = left + (right - left) / 2;
 mergeSort(arr, left, mid);
 mergeSort(arr, mid + 1, right);
 merge(arr, left, mid, right);
 }
}
int main() {
 int n;
 scanf("%d", &n);
 long long *salaries = (long long *)malloc(n * sizeof(long lo
 for (int i = 0; i < n; i++) scanf("%lld", &salaries[i]);
 mergeSort(salaries, 0, n - 1);
 for (int i = 0; i < n; i++) {
 printf("%lld", salaries[i]);
 if (i != n - 1) printf(" ");
 }
 printf("\n");
 long long lowest = salaries[0];
 long long highest = salaries[n - 1];
 long long difference = highest - lowest;
 printf("Lowest Salary: %lld\n", lowest);
 printf("Highest Salary: %lld\n", highest);
 printf("Difference: %lld\n", difference);
 free(salaries);
 return 0;
}
```

Console I/O AI Agent 2 ms

STDIN

```
7
45000 25000 70000 50000 30000 90000 60000
```

Output:

```
25000 30000 45000 50000 60000 70000 90000
Lowest Salary: 25000
Highest Salary: 90000
Difference: 65000
```

Main.c

```c
#include <stdio.h>
#include <stdlib.h>
void swap(int *a, int *b) {
 int temp = *a;
 *a = *b;
 *b = temp;
}
int partition(int arr[], int low, int high) {
 int pivot = arr[high];
 int i = low - 1;
 for (int j = low; j < high; j++) {
 if (arr[j] <= pivot) {
 i++;
 swap(&arr[i], &arr[j]);
 }
 }
 swap(&arr[i + 1], &arr[high]);
 return i + 1;
}
void quickSort(int arr[], int low, int high) {
 if (low < high) {
 int pi = partition(arr, low, high);
 quickSort(arr, low, pi - 1);
 quickSort(arr, pi + 1, high);
 }
}
int main() {
 int n;
 scanf("%d", &n);
 int *timings = (int *)malloc(n * sizeof(int));
 for (int i = 0; i < n; i++) scanf("%d", &timings[i]);
 quickSort(timings, 0, n - 1);
 for (int i = 0; i < n; i++) {
 printf("%d", timings[i]);
 if (i != n - 1) printf(" ");
 }
 printf("\n");
 free(timings);
 return 0;
}
```

Console I/O AI Agent 2 ms

STDIN

```
6
55 49 62 58 45 51
```

Output:

```
45 49 51 55 58 62
```

Main.c

```
ude <stdio.h>
ude <stdlib.h>
swap(long long *a, long long *b) {
 long temp = *a;
 *b;
 temp;

artition(long long arr[], int low, int high) {
 long pivot = arr[high];
 = low - 1;
int j = low; j < high; j++) {
rr[j] <= pivot) {

&arr[i], &arr[j]);


&arr[i + 1], &arr[high]);
n i + 1;

quickSort(long long arr[], int low, int high) {
low < high) {
i = partition(arr, low, high);
Sort(arr, low, pi - 1);
Sort(arr, pi + 1, high);


ain() {
;
("%d", &n);
 long *ids = (long long *)malloc(n * sizeof(long long));
int i = 0; i < n; i++) scanf("%lld", &ids[i]);
Sort(ids, 0, n - 1);
int i = 0; i < n; i++) {
f("%lld", ids[i]);
 != n - 1) printf(" ");

f("\n");
ids);
n 0;
```

Console | I/O | AI Agent | 2 ms

STDIN

```
5
1025 1001 1050 1010 1005
```

Output:

```
1001 1005 1010 1025 1050
```

Main.c

```c
#include <stdio.h>
#include <stdlib.h>
void swap(long long *a, long long *b) {
 long long temp = *a;
 *a = *b;
 *b = temp;
}
// Partition for ascending order first, we'll reverse for des
int partition(long long arr[], int low, int high) {
 long long pivot = arr[high];
 int i = low - 1;
 for (int j = low; j < high; j++) {
 if (arr[j] <= pivot) {
 i++;
 swap(&arr[i], &arr[j]);
 }
 }
 swap(&arr[i + 1], &arr[high]);
 return i + 1;
}
void quickSort(long long arr[], int low, int high) {
 if (low < high) {
 int pi = partition(arr, low, high);
 quickSort(arr, low, pi - 1);
 quickSort(arr, pi + 1, high);
 }
}
int main() {
 int n;
 scanf("%d", &n);
 long long *sales = (long long *)malloc(n * sizeof(long long)
 for (int i = 0; i < n; i++) scanf("%lld", &sales[i]);
 quickSort(sales, 0, n - 1);
 // Print in descending order (reverse of ascending sorted ar
 for (int i = n - 1; i >= 0; i--) {
 printf("%lld", sales[i]);
 if (i != 0) printf(" ");
 }
 printf("\n");
 // Top 3 highest sales (last 3 elements of ascending sorted
 long long top1 = sales[n - 1];
 long long top2 = sales[n - 2];
 long long top3 = sales[n - 3];
 printf("Top 3: %lld %lld %lld\n", top1, top2, top3);
 double average = (top1 + top2 + top3) / 3.0;
 // Print average without unnecessary decimals if it's a whol
 if (average == (long long)average) {
 printf("Average: %lld\n", (long long)average);
 } else {
 printf("Average: %.2f\n", average);
 }
 free(sales);
 return 0;
}
```

Console | I/O | AI Agent | 2 ms

STDIN

```
6
1200 5000 3000 2500 7000 4500
```

Output:

```
7000 5000 4500 3000 2500 1200
Top 3: 7000 5000 4500
Average: 5500
```